

CS355 Lab 2 — Batch A (14:00–15:25)

Language: Scheme (DrRacket, `#lang sicp`)

Q1 (1 mark) — `curried-avg3` (file name: q1.scm)

Write a **curried** procedure `curried-avg3` that takes three numbers one at a time (via nested `lambdas`, not Racket's multi-argument `define` shorthand) and returns their average.

```
((curried-avg3 3) 6) 9) => 6
((curried-avg3 0) 0) 0) => 0
((curried-avg3 10) 20) 30) => 20
```

Q2 (2 marks) — Church encodings, extended (file name: q2.scm)

You've seen how booleans and numbers can be built purely out of functions in lambda calculus. Assume the following definitions are already available to you — copy this block into your file as-is, don't redefine them:

```
(define ctrue (lambda (t) (lambda (f) t)))
(define cfalse (lambda (t) (lambda (f) f)))
(define cif (lambda (cb) (lambda (t) (lambda (f) ((cb t) f))))))
(define czero (lambda (f) (lambda (x) x)))
(define csucc (lambda (n) (lambda (f) (lambda (x) (f ((n f) x))))))
(define church->int (lambda (n) ((n (lambda (k) (+ k 1))) 0)))
```

Firstly, make sure you understand how these procedures work. Next, your job is to build **new** things on top of these — using only `lambda`, function application, and the definitions above (`church->int` is for testing purposes only; don't use it, or Scheme's real `if`, inside the two answers below — they should stay purely in "Church form").

(a) `church-xor` (1 mark). Write a curried procedure `church-xor` that takes two Church booleans and returns a Church boolean that is "true" if and only if *exactly one* of them is "true" (i.e., logical XOR). You're free to define your own helper procedures if useful (a "church-not" that flips a Church boolean might come in handy).

```
((church-xor ctrue) cfalse) 'yes) 'no) ; behaves like ctrue -> 'yes
((church-xor ctrue) ctrue) 'yes) 'no) ; behaves like cfalse -> 'no
```

(Formally: `((church-xor ctrue) cfalse)` would be a procedure that, when applied to `'yes`, returns a procedure that when applied to `'no`, returns `'yes` — the standard way we "read" a Church boolean,

same as with `ctrue/cfalse` themselves.)

(b) `church-is-zero?` (1 mark). Write `church-is-zero?`, which takes a Church numeral and returns a Church boolean: "true" if the numeral represents 0, "false" otherwise. Hint: think about what happens (or doesn't happen) when you apply a function to a value "zero times" versus "at least once."

```
(church-is-zero? czero)           ; behaves like ctrue  
(church-is-zero? (csucc czero)) ; behaves like cfalse
```

Once you're done, feel free to also try Batch B's Q1–Q2 for extra practice — same topics, different problems.